Universidad Nacional Autónoma de México

Laboratorio Virtual de Óptica

Documentación técnica de los módulos de simulación

Descripción de funciones, definiciones físicas y
funcionamiento del código

Módulo I	Reflexión y Refracción
Módulo II	Arreglo de Lentes
Módulo III	Interferencia
Módulo IV	Difracción
Módulo V	Polarización

Python · Streamlit · NumPy · Matplotlib

Índice general

1 Introducción y estructura del proyecto **3**

1.1 Archivos que componen la aplicación . . . 3

1.2 Ejecución . . . 3

1.3 Patrón de funcionamiento común de los módulos . . . 3

1.4 Conceptos de Streamlit utilizados . . . 4

2 Biblioteca matemática: `utils_math.py` **4**

2.1 Reflexión, refracción y Fresnel (usadas en el Módulo I) . . . 4

2.2 Óptica geométrica matricial (usadas en el Módulo II) . . . 5

2.3 Interferencia (usadas en el Módulo III) . . . 6

2.4 Difracción de Fraunhofer (usadas en el Módulo IV) . . . 7

2.5 Polarización y formalismo de Jones (usadas en el Módulo V) . . . 7

3 Módulo I: Reflexión y Refracción **9**

3.1 Definiciones físicas . . . 9

3.2 Controles . . . 9

3.3 Funcionamiento del código . . . 9

4 Módulo II: Arreglo de Lentes **10**

4.1 Definiciones . . . 10

4.2 Controles . . . 10

4.3 Funcionamiento del código . . . 11

5 Módulo III: Interferencia **11**

5.1 Definiciones . . . 11

5.2 Experimento de Young . . . 12

5.3 Interferómetro de Michelson . . . 12

6 Módulo IV: Difracción **12**

6.1 Definiciones . . . 13

6.2 Controles . . . 13

6.3 Funcionamiento del código . . . 13

7 Módulo V: Polarización **13**

7.1 Definiciones . . . 14

7.2 Controles 14
7.3 Funcionamiento del código 14

8 Observaciones técnicas para mantenimiento 15

9 Resumen de funciones por módulo 16

1 Introducción y estructura del proyecto

El Laboratorio Virtual de Óptica es una aplicación web interactiva desarrollada en Python con **Streamlit**. Está compuesta por una página de bienvenida, cinco módulos de simulación (uno por área temática de la óptica) y una biblioteca matemática común. Cada módulo permite modificar parámetros físicos mediante controles en la barra lateral y observar de inmediato el resultado numérico y gráfico.

1.1 Archivos que componen la aplicación

Archivo	Función
`app.py`	Página de inicio. Configura el título (*Simulador de Óptica UNAM*) y da la bienvenida; la navegación entre módulos la proporciona el menú lateral de Streamlit.
`utils_math.py`	Motor matemático. Contiene todas las funciones de cálculo (sin interfaz gráfica), reutilizadas por los módulos.
`1_Módulo_I_Reflexión_y_Refracción.py`	Ley de Snell, coeficientes de Fresnel y principio de Fermat.
`2_Módulo_II_Arreglo_de_Lentes.py`	Banco óptico virtual: trazado de rayos paraxial y formalismo matricial ABCD.
`3_Módulo_III_Interferencia.py`	Doble rendija de Young e interferómetro de Michelson.
`4_Módulo_IV_Difracción.py`	Difracción de Fraunhofer por rendija simple y apertura rectangular.
`5_Módulo_V_Polarización.py`	Formalismo de Jones: polarizadores, retardadores y elipse de polarización.

Sobre la numeración. En este documento los módulos se numeran como en las páginas de la aplicación (Módulo I = Reflexión y Refracción, Módulo II = Arreglo de Lentes). Los comentarios internos de `utils_math.py` usan una numeración distinta (allí «Módulo II» es Reflexión/Fresnel y «Módulo I» son las matrices ABCD); conviene unificarlos (véase la sección 8).

1.2 Ejecución

La aplicación se inicia desde la carpeta del proyecto con `streamlit run app.py`. Streamlit detecta los archivos de módulos (en la estructura multipágina, dentro de la carpeta `pages/`) y los ordena en el menú lateral según el prefijo numérico de su nombre. Cada módulo importa las funciones que necesita con `from utils_math import ...`.

1.3 Patrón de funcionamiento común de los módulos

Todos los módulos siguen la misma arquitectura en tres capas:

1. **Entrada:** los controles de la barra lateral (`st.sidebar.slider`, `radio`, `selectbox`, formularios) producen los valores de los parámetros físicos.
2. **Cálculo:** se llama a una o más funciones de `utils_math.py`, que devuelven números o arreglos de NumPy. Los módulos no repiten la física; solo la invocan.
3. **Salida:** se presentan ecuaciones (`st.latex`), métricas (`st.metric`), mensajes (`st.info`, `st.success`, `st.warning`) y gráficas de Matplotlib (`st.pyplot`), organizadas con `st.columns` y `st.tabs`.

1.4 Conceptos de Streamlit utilizados

Re-ejecución del script. Cada vez que el usuario mueve un control, Streamlit vuelve a ejecutar el archivo completo de arriba hacia abajo. Por eso los resultados se actualizan al instante sin código de eventos.

`st.session_state`. Diccionario persistente entre re-ejecuciones. Los módulos II y V lo usan para recordar la lista de elementos ópticos que el usuario ha añadido.

Formularios (`st.form`). Agrupan controles y solo envían sus valores al presionar el botón de envío, evitando recálculos en cada movimiento del deslizador.

`st.rerun()`. Fuerza una nueva ejecución inmediata del script; se usa tras modificar `session_state` para que la interfaz refleje el cambio.

`st.empty()`. Contenedor que puede sobrescribirse repetidamente; el Módulo V lo emplea para animar la gráfica.

2 Biblioteca matemática: `utils_math.py`

Este archivo reúne 17 funciones puras, agrupadas por tema. Todas usan únicamente NumPy y no dependen de Streamlit, lo que permite probarlas de forma independiente. Salvo que se indique otra cosa, los ángulos se reciben en **grados** y se convierten internamente a radianes.

2.1 Reflexión, refracción y Fresnel (usadas en el Módulo I)

`calcular_snell(n1, n2, theta_i_deg)`

Propósito. Obtiene el ángulo de refracción mediante la ley de Snell,

$$n_1 \operatorname{sen}\theta_i = n_2 \operatorname{sen}\theta_t.$$

Parámetros. `n1`, `n2`: índices de refracción de los medios de incidencia y transmisión; `theta_i_deg`: ángulo de incidencia.
Retorna. Una tupla `(theta_t_deg, tir, theta_critico)`:

- `theta_t_deg`: ángulo refractado en grados, o `None` si hay reflexión total interna.
- `tir`: `True` si $\operatorname{sen}\theta_t > 1$ (reflexión total interna).
- `theta_critico`: ángulo crítico $\theta_c = \arcsen(n_2/n_1)$, definido solo cuando $n_1 > n_2$; en otro caso `None`.

`calcular_fresnel(n1, n2, theta_i_deg)`

Propósito. Calcula la reflectancia R y la transmitancia T de una interfaz dieléctrica para las polarizaciones s (perpendicular al plano de incidencia) y p (paralela), y su promedio para luz no polarizada.

Fundamento. Con $\cos\theta_i$ y $\cos\theta_t$ (el segundo obtenido de Snell), los coeficientes de amplitud son

$$r_s = \frac{n_1\cos\theta_i - n_2\cos\theta_t}{n_1\cos\theta_i + n_2\cos\theta_t}, \qquad r_p = \frac{n_1\cos\theta_t - n_2\cos\theta_i}{n_1\cos\theta_t + n_2\cos\theta_i},$$

y las reflectancias de potencia son $R_s = r_s^2$ y $R_p = r_p^2$. La transmitancia se obtiene por conservación de la energía, $T = 1-R$ (se supone un medio sin absorción). Para luz no polarizada, $R = (R_s + R_p)/2$.

Casos especiales. Si $\operatorname{sen}\theta_t \geq 1$ (reflexión total interna) devuelve $R = 1$ y $T = 0$ en todas las componentes. En incidencia normal ($\theta_i = 0$) se usa la expresión directa $R = \left(\frac{n_1-n_2}{n_1+n_2}\right)^2$, idéntica para s y p.

Retorna. `(R_s, R_p, R_prom, T_s, T_p, T_prom)`.

`analizar_camino_optico(n1, n2, x_A, y_A, x_B, y_B, x_interfaz, resolucion=200)`

Propósito. Verificar numéricamente el principio de Fermat. Un rayo parte del punto $A = (x_A, y_A)$ en el medio 1, cruza la frontera ($y = 0$) en el punto $(x, 0)$ y llega a $B = (x_B, y_B)$ en el medio 2. El camino óptico (LCO) es

$$\mathrm{LCO}(x) = n_1\sqrt{(x - x_A)^2 + y_A^2} + n_2\sqrt{(x_B - x)^2 + y_B^2}.$$

Qué hace. (1) Evalúa el LCO para el punto de cruce elegido `x_interfaz`. (2) Calcula la curva completa $\mathrm{LCO}(x)$ en `resolucion` puntos entre $x_A - 2$ y $x_B + 2$. (3) Localiza el mínimo con `argmin`. La condición $d\mathrm{LCO}/dx = 0$ equivale exactamente a la ley de Snell, de modo que el mínimo de la curva coincide con la trayectoria real.

Retorna. `(opl_puntual, x_array, opl_array, x_min, opl_min)`: LCO en el punto elegido, la curva, y la posición y valor del mínimo.

2.2 Óptica geométrica matricial (usadas en el Módulo II)

El formalismo ABCD describe un rayo paraxial mediante el vector columna $(y, \theta)^T$: altura sobre el eje óptico y ángulo con respecto a él (en radianes). Cada elemento óptico es una matriz 2×2 y la propagación a través de un sistema se obtiene multiplicando matrices.

`matriz_translacion(d)`

Matriz de propagación libre una distancia d: $T(d) = \begin{pmatrix} 1 & d \\ 0 & 1 \end{pmatrix}$.

`matriz_lente_delgada(f)`

Matriz de una lente delgada de distancia focal f (positiva si es convergente, negativa si es divergente): $L(f) = \begin{pmatrix} 1 & 0 \\ -1/f & 1 \end{pmatrix}$. Si $f = 0$ se sustituye por 10^{-9} para evitar la división entre cero.

`matriz_espejo_esferico(R)`

Matriz de un espejo esférico de radio R (equivale a una lente de focal $R/2$ en el eje desplegado): $M(R) = \begin{pmatrix} 1 & 0 \\ -2/R & 1 \end{pmatrix}$. Igualmente protege el caso $R = 0$.

`trazar_rayo_sistema(y0, theta0_deg, componentes, d_final=20.0)`

Propósito. Sigue un rayo a través de una secuencia de elementos y devuelve los puntos necesarios para dibujarlo.
Parámetros. `y0`: altura inicial; `theta0_deg`: ángulo inicial; `componentes`: lista de tuplas `(tipo, valor, d)`, donde `tipo` es `'lente'` o `'espejo'`, `valor` es la focal (o el radio) y `d` la distancia desde el elemento anterior (para el primero, desde el objeto); `d_final`: propagación adicional tras el último elemento.
Algoritmo. Para cada componente: si $d > 0$ propaga el rayo con $T(d)$ y guarda el punto (z, y); después aplica la matriz del elemento (cambia el ángulo, no la altura). Al terminar propaga `d_final` y guarda el último punto.
Retorna. `(posiciones_z, alturas_y, rayo_actual)`: las coordenadas de los vértices de la trayectoria poligonal y el vector $(y, \theta)^T$ final.

`obtener_matriz_sistema(componentes)`

Calcula la matriz equivalente del sistema completo, $M = M_n \cdots M_2 M_1$, aplicando en cada paso la traslación previa (si $d > 0$) y luego la matriz del elemento. No incluye la propagación final `d_final`. A partir de sus elementos $\begin{pmatrix} A & B \\ C & D \end{pmatrix}$ el Módulo II obtiene la focal equivalente $f_{eq} = -1/C$.

2.3 Interferencia (usadas en el Módulo III)

`calcular_patron_young(longitud_onda_nm, d_mm, L_cm, tamano_pantalla_cm, resolucion=500)`

Propósito. Genera el patrón bidimensional de irradiancia de dos fuentes puntuales, sin aproximación paraxial.
Fundamento. Las rendijas se ubican en $(0, \pm d/2, 0)$ y la pantalla en $z = L$. Para cada punto (X, Y) se calculan las distancias exactas

$$r_{1,2} = \sqrt{X^2 + (Y \mp d/2)^2 + L^2}, \qquad \Delta\phi = \frac{2\pi}{\lambda}(r_2 - r_1), \qquad \frac{I}{I_{max}} = \cos^2\left(\frac{\Delta\phi}{2}\right).$$

Unidades. Recibe λ en nm, d en mm, L y el tamaño de ventana en cm, y los convierte a metros.
Retorna. `X, Y` (mallas en metros) e `intensidad` (arreglo normalizado en $[0, 1]$).

`calcular_patron_michelson(longitud_onda_nm, delta_d_um, f_lente_cm, tamano_pantalla_cm, resolucion=500)`

Propósito. Calcula las franjas de igual inclinación (anillos de Haidinger) del interferómetro de Michelson.
Fundamento. Un punto de la pantalla a distancia radial $R = \sqrt{X^2 + Y^2}$ del eje corresponde, a través de la lente proyectora de focal f, a un ángulo $\theta = \arctan(R/f)$. La diferencia de camino entre los brazos desplazados Δd es $2\Delta d \cos\theta$, así que

$$\Delta\phi = \frac{4\pi}{\lambda}\,\Delta d\cos\theta, \qquad \frac{I}{I_{max}} = \cos^2\!\left(\frac{\Delta\phi}{2}\right).$$

Retorna. `X, Y, intensidad`, igual que la función de Young. Aquí Δd se recibe en μm y f en cm.

2.4 Difracción de Fraunhofer (usadas en el Módulo IV)

En el límite de campo lejano, la irradiancia de una apertura rectangular de lados $a \times b$ es el producto de dos funciones sinc^2. NumPy define $\mathrm{np.sinc}(u) = \mathrm{sen}(\pi u)/(\pi u)$, por lo que el argumento se escribe como $a\,\mathrm{sen}\,\theta_x/\lambda$.

`calcular_difraccion_rendija(longitud_onda_nm, ancho_a_mm, L_cm, tamano_pantalla_cm, resolucion=500)`

Rendija simple vertical de ancho a. Con $\rho = \sqrt{X^2 + Y^2 + L^2}$ y $\mathrm{sen}\,\theta_x = X/\rho$:

$$I(X) = I_0\,\mathrm{sinc}^2\!\left(\frac{a\ \mathrm{sen}\,\theta_x}{\lambda}\right).$$

La intensidad depende solo de X, por lo que el patrón consiste en franjas verticales. Retorna `X, Y, intensidad`.

`calcular_difraccion_rectangular(longitud_onda_nm, ancho_a_mm, altura_b_mm, L_cm, tamano_pantalla_cm, resolucion=500)`

Apertura rectangular $a \times b$. Con $\mathrm{sen}\,\theta_y = Y/\rho$:

$$I(X,Y) = I_0\,\mathrm{sinc}^2\!\left(\frac{a\ \mathrm{sen}\,\theta_x}{\lambda}\right)\mathrm{sinc}^2\!\left(\frac{b\ \mathrm{sen}\,\theta_y}{\lambda}\right).$$

Retorna `X, Y, intensidad`.

2.5 Polarización y formalismo de Jones (usadas en el Módulo V)

En el formalismo de Jones, un estado de polarización totalmente polarizado se representa por un vector complejo $\vec{E} = (E_x, E_y)^T$ y cada elemento óptico por una matriz 2×2. El campo de salida es $\vec{E}_{sal} = J\,\vec{E}_{ent}$, y para varios elementos en serie se multiplican las matrices en orden inverso al de paso de la luz. La irradiancia relativa es $I = |E_x|^2 + |E_y|^2$.

`vector_jones_inicial(estado)`

Devuelve el vector de Jones normalizado, como arreglo complejo de 2×1, para el estado indicado:

Estado	**Vector**
Lineal Horizontal	$(1,\,0)^T$
Lineal Vertical	$(0,\,1)^T$
Lineal $+45°$	$\frac{1}{\sqrt{2}}(1,\,1)^T$
Circular Derecha	$\frac{1}{\sqrt{2}}(1,\,-i)^T$
Circular Izquierda	$\frac{1}{\sqrt{2}}(1,\,i)^T$

Cualquier otro texto se interpreta como lineal horizontal.

`matriz_polarizador_lineal(theta_deg)`

Polarizador lineal con eje de transmisión a un ángulo θ respecto a la horizontal:

$$P(\theta) = \begin{pmatrix} \cos^2\theta & \operatorname{sen}\theta\cos\theta \\ \operatorname{sen}\theta\cos\theta & \operatorname{sen}^2\theta \end{pmatrix}.$$

Proyecta el campo sobre su eje de transmisión (de ahí la ley de Malus).

`matriz_retardador(theta_deg, gamma_deg)`

Retardador (lámina de onda) con eje rápido a un ángulo θ y retardo de fase γ entre sus ejes. Se construye rotando al sistema del eje propio, aplicando el retardo y rotando de regreso:

$$W(\theta,\gamma) = R(-\theta)\begin{pmatrix} e^{i\gamma/2} & 0 \\ 0 & e^{-i\gamma/2} \end{pmatrix} R(\theta), \qquad R(\theta) = \begin{pmatrix} \cos\theta & \operatorname{sen}\theta \\ -\operatorname{sen}\theta & \cos\theta \end{pmatrix}.$$

Con $\gamma = 90°$ se obtiene una lámina de cuarto de onda ($\lambda/4$) y con $\gamma = 180°$ una de media onda ($\lambda/2$).

`curva_elipse_polarizacion(Ex, Ey, resolucion=300)`

Genera la trayectoria que describe la punta del vector de campo eléctrico real en el plano transversal durante un periodo. Toma la parte real de los fasores rotando en el tiempo ($e^{-i\omega t}$, con $\omega t \in [0, 2\pi]$):

$$E_x(t) = \operatorname{Re}\{E_x e^{-i\omega t}\}, \qquad E_y(t) = \operatorname{Re}\{E_y e^{-i\omega t}\}.$$

Retorna dos arreglos `(Ex_t, Ey_t)` que, graficados uno contra otro, trazan una recta (polarización lineal), un círculo (circular) o una elipse.

`jones_a_euler_latex(Jx, Jy)`

Convierte las componentes complejas del campo de salida a una cadena en LaTeX con notación de Euler, $A\,e^{i\varphi}$, lista para `st.latex`. Una componente de amplitud menor que 10^{-4} se escribe como `0`, y una de fase nula se escribe solo con su amplitud. Fuerza la conversión a `complex` nativo antes de operar.

3 Módulo I: Reflexión y Refracción

Archivo: `1_Módulo_I_Reflexión_y_Refracción.py` **Funciones usadas:** `calcular_snell`, `calcular_fresnel`, `analizar_camino_optico`.

Qué hace. Muestra cómo un rayo incide sobre una interfaz entre dos medios, cuánta energía se refleja y se transmite (Fresnel), y demuestra que la trayectoria real es la que minimiza el camino óptico (Fermat).

3.1 Definiciones físicas

Ley de Snell. $n_1 \operatorname{sen}\theta_i = n_2 \operatorname{sen}\theta_t$; relaciona los ángulos medidos respecto a la normal.

Ley de la reflexión. El ángulo reflejado es igual al de incidencia, $\theta_r = \theta_i$.

Coeficientes de Fresnel. Dan la fracción de energía reflejada (R) y transmitida (T), distinta para cada polarización.

Ángulo de Brewster. Ángulo $\theta_B = \arctan(n_2/n_1)$ en el que $R_p = 0$: la luz reflejada queda polarizada en s.

Principio de Fermat. La luz recorre la trayectoria de camino óptico estacionario (mínimo): LCO $= \sum n_i L_i$.

3.2 Controles

Control	Significado	Rango (inicial)
Índice n_1	Medio de incidencia	1.0–3.0 (1.0)
Índice n_2	Medio de transmisión; su mínimo es n_1, de modo que siempre $n_2 \geq n_1$ y no ocurre reflexión total interna	n_1–3.0 (1.5)
Ángulo θ_1	Ángulo de incidencia para Snell/Fresnel	0–89.99° (45°)
Punto de incidencia x	Posición del cruce del rayo de prueba en la frontera (Fermat)	0–8 cm (4)

3.3 Funcionamiento del código

1. Lee los controles y llama a `calcular_snell` para obtener θ_t.
2. Fija los puntos de Fermat: $A = (0, 4)$ y $B = (8, -4)$ cm, y llama a `analizar_camino_optico` con el x elegido.
3. La pantalla se divide en dos columnas.
   - **Columna izquierda (teoría y números):** ecuación de Snell, ángulo de refracción, métricas de R y T (promedio, y por componentes s y p) a partir de `calcular_fresnel`, y una alerta didáctica cuando $R_p < 0.001$ con $\theta_1 > 0$ (se alcanzó el ángulo de Brewster). Debajo se muestran el LCO actual y el mínimo absoluto.
   - **Mensaje de estado de Fermat:** si $|x - x_{min}| < 0.15$ cm informa que se encontró la trayectoria real; en caso contrario indica cuánto excede el LCO actual al mínimo y hacia dónde mover el deslizador.
   - **Columna derecha (gráficas)**, en dos pestañas.

Pestaña 1: Trazado de rayos (Snell). Dibuja la frontera, la normal, el rayo incidente (rojo), el reflejado (naranja, discontinuo) y el refractado (verde), con arcos que marcan θ_i, θ_r y θ_t. El grosor y la opacidad de los rayos reflejado y transmitido son proporcionales a R y T, de modo que se *ve* el reparto de energía.

Pestaña 2: Minimización (Fermat). A la izquierda, el camino geométrico de prueba de A a B pasando por x; los ángulos θ_i y θ_t se calculan de la geometría y se evalúa Fresnel con ese ángulo. A la derecha, la curva $\mathrm{LCO}(x)$ con un punto rojo en la posición actual y una línea verde en el mínimo. Cuando el punto rojo coincide con el mínimo, los ángulos geométricos cumplen la ley de Snell.

Los puntos A y B de Fermat son fijos; por eso el ángulo de la pestaña 2 depende de x y no del control θ_1 de la barra lateral. El código lo advierte con un mensaje informativo en la propia pantalla.

4 Módulo II: Arreglo de Lentes

Archivo: `2_Módulo_II_Arreglo_de_Lentes.py` **Funciones usadas:** `trazar_rayo_sistema`, `obtener_matriz_sistema`, y la función propia `dibujar_flecha`.

Qué hace. Es un banco óptico virtual: el usuario coloca un objeto y añade lentes una por una; el programa traza rayos, localiza la imagen final (real o virtual), calcula su aumento y presenta la matriz ABCD del sistema con su focal equivalente.

4.1 Definiciones

Aproximación paraxial. Rayos cercanos al eje y con ángulos pequeños, de modo que $\operatorname{sen}\theta \approx \tan\theta \approx \theta$. Hace lineal la óptica y permite usar matrices.

Imagen real / virtual. Real: los rayos realmente convergen en el punto imagen (se dibuja con trazo continuo, naranja). Virtual: solo sus prolongaciones hacia atrás convergen (trazo discontinuo, morado).

Aumento lateral. $m = y_{img}/y_{obj}$; negativo si la imagen está invertida.

Focal equivalente. Focal de la lente única equivalente a todo el sistema, $f_{eq} = -1/C$.

Planos principales. Planos de referencia a partir de los cuales el sistema se comporta como una lente delgada de focal f_{eq}.

Sistema afocal. Sistema con $C = 0$ (focal equivalente infinita), por ejemplo un telescopio colimador.

4.2 Controles

- **Configuración de la fuente** (formulario): modo de emisión, *Formación de Imagen (Objeto)* o *Rayo Único*. En el primer modo se fija la altura del objeto (0.5–5 cm); en el segundo, la altura y_0 y el ángulo θ_0 del rayo ($\pm 15°$). Si ya hay lentes, también se ajusta la distancia del objeto a la primera lente (1–100 cm).
- **Agregar componente** (formulario): focal f en cm (positiva convergente, negativa divergente) y distancia al elemento anterior. El botón añade una tupla `('lente', f, d)` a `st.session_state.componentes`.
- **Limpiar banco óptico:** vacía la lista.

- **Pantalla de proyección:** distancia de propagación adicional `d_final` (10–100 cm).

4.3 Funcionamiento del código

1. **Memoria.** La lista de lentes se guarda en `st.session_state.componentes`; el objeto está siempre en $z = 0$ y la primera lente en $z = d_1$.
2. **Panel izquierdo.** Lista las lentes (tipo, focal, posición relativa) y, con `obtener_matriz_sistema`, muestra la matriz M_{sys}, la focal equivalente y, en un desplegable, las posiciones de los planos principales
$$h_1 = \frac{D-1}{C}, \qquad h_2 = \frac{A-1}{C}.$$
Si $C = 0$ muestra una advertencia de sistema afocal.
3. **Rayos.** En modo *Rayo Único* se traza un rayo con `trazar_rayo_sistema`. En modo *Objeto* se trazan tres rayos desde la punta del objeto: uno paralelo al eje ($\theta = 0$), uno dirigido al centro de la primera lente ($\theta = \arctan(-y_0/d_1)$) y un tercero con el doble de ese ángulo.
4. **Elementos.** Cada lente se dibuja como una línea vertical en su posición acumulada z, azul continua si converge y naranja discontinua si diverge, con su etiqueta L_i y su focal.
5. **Localización de la imagen.** Se toman los *últimos tramos* de los dos primeros rayos (después de la última lente), cada uno como recta $y = mz + c$, y se calcula su intersección:
$$z_{img} = \frac{c_2 - c_1}{m_1 - m_2}, \qquad y_{img} = m_1 z_{img} + c_1.$$
Si z_{img} queda antes de la última lente, la imagen es virtual y se dibujan las extensiones hacia atrás; si no, es real. Si las pendientes son prácticamente iguales (rayos paralelos) no se forma imagen puntual.
6. **Datos analíticos.** Debajo de la gráfica se muestran el tipo de imagen, su posición, su altura y el aumento $m = y_{img}/y_0$.
7. **Función `dibujar_flecha(ax, z, y, tipo)`.** Dibuja la flecha del objeto (verde), de la imagen real (naranja) o de la imagen virtual (morada, semitransparente y discontinua) con su etiqueta, que se coloca arriba o abajo según la flecha esté derecha o invertida.

5 Módulo III: Interferencia

Archivo: `3_Módulo_III_Interferencia.py` **Funciones usadas:** `calcular_patron_young`, `calcular_patron_michelson`.

Qué hace. Simula dos mecanismos de interferencia, seleccionables en la barra lateral: la *doble rendija de Young* (división de frente de onda) y el *interferómetro de Michelson* (división de amplitud). Presenta el interferograma 2D, un perfil 1D y un análisis cuantitativo.

5.1 Definiciones

Interferencia. Superposición de ondas coherentes; la irradiancia resultante depende de su diferencia de fase $\Delta\phi$.

Condiciones de máximo y mínimo. Máximo cuando $\Delta\phi = 2\pi m$ (diferencia de camino $m\lambda$); mínimo cuando $\Delta\phi = (2m+1)\pi$.

Franjas de igual inclinación (Haidinger). Anillos concéntricos que aparecen cuando la diferencia de camino depende del ángulo de observación, como en el Michelson con fuente extendida.

Sensibilidad $\lambda/2$. Al desplazar un espejo del Michelson $\lambda/2$, la diferencia de camino cambia λ y el centro pasa de brillante a oscuro (o viceversa) un ciclo completo.

5.2 Experimento de Young

Controles. λ (380–750 nm, 532), separación d (0.01–0.50 mm, 0.10), distancia a la pantalla L (10–500 cm, 100) y ventana de observación (1–20 cm, 5).

Funcionamiento.

1. Se muestra el fundamento teórico: $I(y) = 4I_0 \cos^2(\Delta\phi/2)$ y $\Delta\phi = \frac{2\pi}{\lambda}(r_2 - r_1)$.
2. Se llama a `calcular_patron_young` y se elige el color según la longitud de onda: azules (< 495 nm), verdes (< 570), naranjas (< 620) y rojos (resto).
3. **Interferograma 2D** con `imshow` y **perfil 1D** tomado de la columna central de la matriz (la línea $X \approx 0$) en función de Y. Como las rendijas están separadas a lo largo de Y, las franjas son horizontales.
4. **Análisis cuantitativo: paraxial frente a exacto.** Se compara la separación entre franjas de la aproximación de ángulos pequeños con la posición exacta del primer máximo:

$$\Delta y \approx \frac{\lambda L}{d}, \qquad y_1 = \sqrt{\frac{\lambda^2 L^2}{d^2 - \lambda^2} + \frac{\lambda^2}{4}}.$$

   La segunda expresión proviene de que el lugar geométrico $r_2 - r_1 = \lambda$ es una hipérbola con focos en las rendijas; resolviéndola en $X = 0$ y $z = L$ se obtiene y_1. El código muestra ambos valores en cm. (El cálculo del error porcentual existe en el código pero su métrica está comentada.)

5.3 Interferómetro de Michelson

Controles. λ (380–750 nm, 632 nm, rojo He–Ne), desplazamiento del espejo Δd (0–50 μm con paso de 0.001, 10), focal de la lente proyectora f (5–50 cm, 20) y ventana (1–20 cm, 10).

Funcionamiento.

1. Se explica el desfase $\Delta\phi = \frac{4\pi}{\lambda}\Delta d \cos\theta$.
2. `calcular_patron_michelson` produce los anillos; se presentan como imagen 2D y como perfil horizontal pasando por el centro.
3. **Análisis dinámico:** se muestra la sensibilidad $\lambda/2$ en μm y la *intensidad relativa central*, leída del píxel central de la matriz. En el centro ($\theta = 0$) vale $\cos^2(2\pi\Delta d/\lambda)$: al variar Δd con el deslizador se observa cómo oscila entre 0 y 1 cada $\lambda/2$.

6 Módulo IV: Difracción

Archivo: `4_Módulo_IV_Difracción.py` **Funciones usadas:** `calcular_difraccion_rendija`, `calcular_difraccion_rectangular`.

Qué hace. Simula el patrón de difracción de Fraunhofer (campo lejano) de una rendija simple o de una apertura rectangular, en escala lineal o logarítmica, y cuantifica el ancho del máximo central y la irradiancia del primer máximo secundario.

6.1 Definiciones

Difracción. Desviación y ensanchamiento de la luz al atravesar una apertura de tamaño comparable a λ.

Aproximación de Fraunhofer. Campo lejano: la pantalla está lo bastante lejos como para que los frentes de onda en la apertura se traten como planos.

Máximo central. Región entre los dos primeros mínimos; su semiancho es $x_1 = \lambda L/a$ y su ancho total $W = 2\lambda L/a$.

Máximos secundarios. Franjas mucho menos intensas a los lados; el primero se encuentra en $\approx 1.4303\, x_1$ y su irradiancia es $\approx 0.047\, I_0$ (4.7 %).

6.2 Controles

- **Tipo de apertura:** rendija simple (1D) o rectangular (2D).
- λ (380–750 nm, 532), ancho a (0.01–0.50 mm, 0.08) y, solo en modo rectangular, alto b (0.01–0.50 mm, 0.15). En modo rendija se fija $b = 0$ y no se usa.
- Distancia L (10–500 cm, 100) y ventana (1–10 cm, 3).
- **Escala de intensidad:** lineal o logarítmica.

6.3 Funcionamiento del código

1. Según la geometría se llama a la función de rendija o a la rectangular. El color se elige por longitud de onda, igual que en el Módulo III.
2. **Escalas.** En logarítmica se usa `LogNorm` entre 10^{-3} y 1, y el perfil 1D usa eje y logarítmico, lo que hace visibles los máximos secundarios. En lineal se usa `Normalize` con `vmax=0.4`: el pico central queda saturado para que las franjas laterales brillen más en la imagen.
3. **Gráficas:** proyección 2D y perfil en X (fila central de la matriz).
4. **Análisis cuantitativo.** Calcula $W = 2\lambda L/a$ y la posición de los primeros mínimos $\pm x_1$. Para el primer máximo secundario, primero verifica que $1.4303\, x_1$ quepa en la ventana; si no, muestra «Ampliar ventana». Si cabe, toma el máximo del perfil numérico en la región $x > 1.01\, x_1$ (justo después del primer mínimo) y lo reporta en unidades de I_0.

7 Módulo V: Polarización

Archivo: `5_Módulo_V_Polarización.py` **Funciones usadas:** `vector_jones_inicial`, `matriz_polarizador_lineal`, `matriz_retardador`, `curva_elipse_polarizacion`, y una versión local de `jones_a_euler_latex`.

Qué hace. Permite construir un tren óptico de polarizadores y láminas retardadoras, elegir la polarización inicial y ver el vector de Jones de salida, la irradiancia relativa y la elipse de polarización (con opción de animación).

7.1 Definiciones

Polarización. Orientación y forma de la oscilación del campo eléctrico en el plano transversal a la propagación.

Lineal, circular, elíptica. Según la relación de amplitudes y fases entre E_x y E_y: fase nula o π (lineal), amplitudes iguales y desfase $\pm\pi/2$ (circular), caso general (elíptica).

Polarizador. Elemento que transmite solo la componente del campo a lo largo de su eje.

Lámina retardadora. Material birrefringente que introduce un desfase γ entre las componentes a lo largo de sus ejes rápido y lento.

Extinción. Irradiancia de salida nula (por ejemplo, dos polarizadores cruzados).

7.2 Controles

- **Fuente de luz:** lista desplegable con los cinco estados de `vector_jones_inicial`.
- **Agregar elemento** (formulario): tipo (polarizador lineal o lámina retardadora), ángulo del eje (grados, por defecto 45) y, para retardadores, el retardo: $\lambda/4$ (90°), $\lambda/2$ (180°) o personalizado. Cada elemento se agrega como diccionario a `st.session_state.elementos_polarizacion`.
- **Limpiar tren óptico:** vacía la lista.

7.3 Funcionamiento del código

1. Se obtiene el vector inicial y se recorre la lista en el orden en que se añadieron los elementos. Para cada uno se construye su matriz de Jones y se actualiza el campo, $\vec{E} \leftarrow M\,\vec{E}$.
2. Se extraen E_x, E_y y la irradiancia relativa $I/I_0 = |E_x|^2 + |E_y|^2$ (con $I_0 = 1$ para el vector inicial normalizado).
3. **Columna izquierda:** lista numerada del tren óptico (polarizadores en verde, retardadores en amarillo), el vector de salida en notación de Euler (`jones_a_euler_latex`) y la métrica de irradiancia.
4. **Columna derecha:** la elipse de polarización. La función interna `dibujar_fotograma(t)` dibuja la trayectoria completa en tono tenue y, encima, el vector instantáneo

$$E_x(t) = |E_x|\cos(-t + \arg E_x), \qquad E_y(t) = |E_y|\cos(-t + \arg E_y),$$

   con un punto y una flecha verdes. Si $I < 10^{-4}$ no hay campo que dibujar y se muestra el mensaje «Extinción Total (Oscuridad)».
5. **Animación.** Al presionar «Animar Propagación del Campo» se dibujan 60 fotogramas con $t \in [0, 4\pi]$ (dos periodos) sobre el mismo contenedor `st.empty()`, con una pausa de 0.04 s entre ellos; cada figura se cierra con `plt.close` para liberar memoria, y al final se regresa al estado $t = 0$.

Con la convención $e^{-i\omega t}$ usada en el código, $(1, -i)/\sqrt{2}$ gira en sentido horario visto desde la fuente hacia el observador, que corresponde a la polarización circular derecha según la convención usual en óptica.

8 Observaciones técnicas para mantenimiento

Las siguientes observaciones no impiden el funcionamiento actual; se recogen para mejorar la consistencia y robustez del código.

1. **Numeración de módulos.** Los comentarios de `utils_math.py` rotulan como «Módulo II» a Reflexión/Fresnel y como «Módulo I» a las matrices ABCD, al revés que los archivos de las páginas.
2. **Cadenas con secuencias de escape inválidas.** Textos como `"$\lambda$"`, `"\%"` o `"$\Delta d$"` están en cadenas normales (no `r"..."`); Python $\geq$ 3.12 emite `SyntaxWarning`. Basta anteponer `r` a esas cadenas.
3. **Función duplicada.** `jones_a_euler_latex` existe en `utils_math.py` y también dentro del Módulo V; el módulo usa la local, cuyo formato de fase es distinto (`e^{+0.79i}` frente a `e^{i 0.79}`). Conviene conservar una sola.
4. **Ramas de reflexión total interna.** El Módulo I limita $n_2 \geq n_1$, así que `tir` es siempre `False` y `theta_critico` es `None`; esas ramas de `calcular_snell` y `calcular_fresnel` se conservan para uso general.
5. **Aproximación paraxial en el Módulo II.** El ángulo en grados se convierte a radianes y se usa directamente como pendiente ($\tan\theta \approx \theta$). Para objetos con y_0/d_1 grande, el rayo «al centro de la lente» deja de pasar exactamente por el centro.
6. **Convención de h_2.** Con $h_2 = (A-1)/C$, un valor positivo indica que el plano principal de salida está *antes* del plano de salida (última lente). El plano de entrada es el del objeto, porque la primera traslación (distancia objeto–lente) forma parte de la matriz.
7. **Detección de sistema afocal.** Se prueba `C != 0` con igualdad exacta de punto flotante; un C muy pequeño pero no nulo produce una focal enorme en lugar del aviso de sistema afocal.
8. **Muestreo en Michelson.** Con Δd grande, f pequeña y ventanas amplias, las franjas del borde pueden ser más finas que el paso de la malla de 500 puntos y producir aliasing (patrones espurios).
9. **Intensidad central.** La malla `linspace` de resolución par no contiene exactamente $X = 0$; el «píxel central» es el vecino a $x = 0$, por lo que la lectura es una muy buena aproximación, no el valor exacto.
10. **Validez de Fraunhofer.** La condición de campo lejano requiere que el número de Fresnel $a^2/(\lambda L)$ sea pequeño. Con $a = 0.5$ mm, $L = 10$ cm y $\lambda = 380$ nm vale ≈ 6.6, fuera de ese régimen; con los valores por defecto ($a = 0.08$ mm, $L = 100$ cm, $\lambda = 532$ nm) vale ≈ 0.012.
11. **Memoria de figuras.** Los módulos I a IV no cierran las figuras de Matplotlib tras `st.pyplot`; el Módulo V sí lo hace. Añadir `plt.close(fig)` evita acumulación de memoria en sesiones largas.
12. **Transmitancia.** $T = 1 - R$ supone un medio sin absorción ni dispersión; no aplica a metales ni a medios absorbentes.

9 Resumen de funciones por módulo

Función	Módulo	Salida principal
`calcular_snell`	I	θ_t, indicador TIR, θ_c
`calcular_fresnel`	I	R y T para s, p y promedio
`analizar_camino_optico`	I	LCO puntual, curva LCO(x) y mínimo
`matriz_translacion`	II	Matriz $T(d)$
`matriz_lente_delgada`	II	Matriz $L(f)$
`matriz_espejo_esferico`	II	Matriz $M(R)$
`trazar_rayo_sistema`	II	Trayectoria (z, y) del rayo
`obtener_matriz_sistema`	II	Matriz ABCD total
`calcular_patron_young`	III	Malla (X, Y) e intensidad
`calcular_patron_michelson`	III	Malla (X, Y) e intensidad
`calcular_difraccion_rendija`	IV	Malla (X, Y) e intensidad
`calcular_difraccion_rectangular`	IV	Malla (X, Y) e intensidad
`vector_jones_inicial`	V	Vector de Jones inicial
`matriz_polarizador_lineal`	V	Matriz de Jones del polarizador
`matriz_retardador`	V	Matriz de Jones del retardador
`curva_elipse_polarizacion`	V	Trayectoria $(E_x(t), E_y(t))$
`jones_a_euler_latex`	V	Cadena LaTeX del campo de salida